

Thu, Aug 6 at 9:59 AM

Internationalization (i18n) - explain only which is required

For a **6–7 years Spring developer interview**, you only need the concepts below. You don't need to know every API in depth.

Internationalization (i18n)

What is Internationalization?

Internationalization (i18n) is the process of making an application support **multiple languages and regions** without changing the application code.

Instead of hardcoding text:

```
</> Java
```

```
return "Welcome";
```



You store it in language-specific files:

```
messages.properties  
messages_fr.properties  
messages_hi.properties
```



Spring loads the correct message based on the user's locale.

Why is it needed?

Suppose your application is used in:

- India
- France
- Germany

Instead of building three applications, Spring changes only the displayed text.

Example:

English

Welcome



French

Bienvenue



Hindi

स्वागत है



Main Components (Interview Important)

1. Locale

A **Locale** represents the user's language and country.

Example

`</> Java`



```
Locale.US  
Locale.UK  
Locale.FRANCE  
new Locale("hi", "IN")
```

Examples

```
en_US  
en_GB  
fr_FR  
hi_IN
```



2. MessageSource

`MessageSource` is Spring's interface for reading messages from properties files.

Spring Boot automatically creates a `MessageSource` bean if it finds:

`messages.properties`



Example

</> Java



```
@Autowired
MessageSource messageSource;

String message =
messageSource.getMessage(
    "welcome.message",
    null,
    Locale.US
);
```

Output

Welcome



3. Message Properties Files

Default

messages.properties



French

messages_fr.properties



Hindi

messages_hi.properties



Example

messages.properties

welcome.message=Welcome



messages_fr.properties

welcome.message=Bienvenue



messages_hi.properties

```
welcome.message=स्वागत है
```



Spring automatically selects the correct file based on the locale.

4. LocaleResolver

Determines **which locale should be used** for the current request.

Common implementations:

- `AcceptHeaderLocaleResolver` (default in Spring Boot)
- `SessionLocaleResolver`
- `CookieLocaleResolver`

AcceptHeaderLocaleResolver (Most Important)

Reads the browser's:

```
Accept-Language
```



Example request

```
Accept-Language: fr
```



Spring loads

```
messages_fr.properties
```



5. Using Messages in Controller

```
</> Java
```



```
@GetMapping("/welcome")
public String welcome(Locale locale) {

    return messageSource.getMessage(
        "welcome.message",
        null,
        locale);
}
```

Spring automatically injects the request's locale.

Message Parameters

Property

greeting=Hello {0}



Code

</> Java



```
messageSource.getMessage(  
    "greeting",  
    new Object[]{"John"},  
    Locale.US  
);
```

Output

Hello John



Spring Boot Configuration

Default basename

messages



Custom

</> properties



```
spring.messages.basename=mymessages
```

Interview Questions

What is i18n?

Supporting multiple languages without changing application code.

What is Locale?

Represents language and country.

Example:

```
en_US  
hi_IN  
fr_FR
```



What is MessageSource?

Spring interface that loads text from properties files based on locale.

What is LocaleResolver?

Determines which locale should be used for the current request.

Which LocaleResolver is the default in Spring Boot?

AcceptHeaderLocaleResolver .

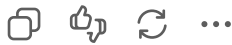
How does Spring know which language to display?

It checks the request's locale (typically from the `Accept-Language` HTTP header), resolves the appropriate locale, and loads the matching `messages_{locale}.properties` file.

What you should know for a 6–7 year interview

- ☒ What Internationalization (i18n) is
- ☒ Purpose of `Locale`
- ☒ Purpose of `MessageSource`
- ☒ `messages.properties` and locale-specific property files
- ☒ `LocaleResolver` and its common implementations
- ☒ Default `AcceptHeaderLocaleResolver` in Spring Boot
- ☒ Parameterized messages (`{0}` , `{1}`)
- ☒ Basic Spring Boot configuration (`spring.messages.basename`)

You generally do not need advanced customization of `MessageSource`, custom locale resolvers, or deep internals unless the role specifically focuses on framework development or localization.

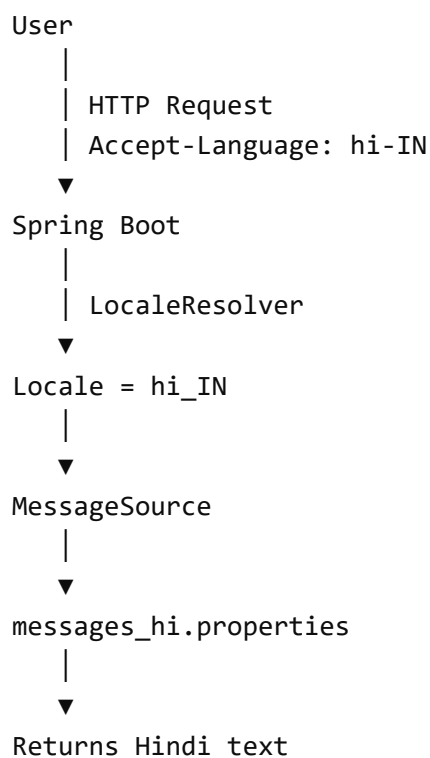


Thu, Aug 6 at 12:08 PM

how do application decides I need to display hindi message?

Great question. The application **doesn't decide on its own** to display Hindi. It decides based on the **user's locale**, which is usually determined from the HTTP request.

Flow



Step 1: Browser sends the language

Every browser sends an HTTP header called:

```
</> http
```

Accept-Language

Example:

```
</> http
```

```
Accept-Language: hi-IN
```



or

```
</> http
```

```
Accept-Language: en-US
```



or

```
</> http
```

```
Accept-Language: fr-FR
```



This tells the server:

"I prefer Hindi" or "I prefer English."

Step 2: Spring reads the header

By default, Spring Boot uses `AcceptHeaderLocaleResolver` .

It reads:

```
</> http
```

```
Accept-Language: hi-IN
```



and creates:

```
</> Java
```

```
Locale locale = new Locale("hi", "IN");
```



Step 3: MessageSource loads the correct file

Suppose your project contains:

```
messages.properties
```




```
messages_hi.properties  
messages_fr.properties
```

If locale is:

```
hi_IN
```

Spring searches for:

```
messages_hi.properties
```

If found:

```
</> properties  
welcome.message=स्वागत है
```

Output:

```
स्वागत है
```

What if Hindi file doesn't exist?

Suppose:

```
messages.properties  
messages_fr.properties
```

User requests:

```
</> http  
Accept-Language: hi-IN
```

Spring cannot find:

```
messages_hi.properties
```

So it falls back to the default:

```
messages.properties
```

Output:

Welcome



Can the application change the language manually?

Yes.

Instead of using the browser's language, you can set the locale yourself.

Example:

</> Java



```
Locale locale = new Locale("hi", "IN");

String message = messageSource.getMessage(
    "welcome.message",
    null,
    locale);
```

Now Spring always returns:

स्वागत है



even if the browser language is English.

Real-world example

Imagine an e-commerce website.

1. Your browser language is English.
2. You log in.
3. In your profile, you select **Hindi**.
4. The application stores "hi" in your profile or session.
5. On each request, the application uses the Hindi locale.
6. MessageSource loads:

messages_hi.properties



and all UI messages appear in Hindi.

Interview answer

Q: How does a Spring application decide to display Hindi messages?

Answer:

Spring determines the user's `Locale`, usually from the `Accept-Language` HTTP header using the default `AcceptHeaderLocaleResolver`. If the locale is `hi_IN`, `MessageSource` loads `messages_hi.properties`. If that file is unavailable, Spring falls back to the default `messages.properties`. The locale can also be set programmatically or retrieved from a user's session, cookie, or profile depending on the application's configuration.



ChatGPT can make mistakes. Check important info.